



unyts

Unit Conversion with Directed Graph Networks

Powered by Graph Search Algorithms

Version 0.10.1

March 5, 2026

<https://github.com/ayaranitram/unyts>

Contents

1	Introduction	5
1.1	The Core Innovation	5
1.2	Key Features	5
1.2.1	Pre-Loaded Unit Networks	5
1.2.2	Unit-Aware Calculations	6
1.2.3	Multiple Search Algorithms	6
1.2.4	Dimensional Validation	6
1.3	Use Cases	6
1.4	Design Philosophy	7
2	Installation	8
2.1	Requirements	8
2.1.1	Python Version	8
2.1.2	Core Dependencies	8
2.1.3	Optional Dependencies	8
2.2	Installation Methods	8
2.2.1	Install from PyPI (Recommended)	8
2.2.2	Upgrade to Latest Version	9
2.2.3	Install from Source	9
2.3	Verification	9
2.4	First-Time Setup	9
2.5	Configuration Files	9
2.6	Troubleshooting Installation	10
2.6.1	Missing cloudpickle	10
2.6.2	Import Errors	10
2.6.3	Version Conflicts	10
3	Quick Start	11
3.1	Basic Conversion	11
3.2	Creating Unit-Aware Quantities	11
3.3	Type Checking	12
3.4	Converting Unit Instances	12
3.5	Working with Arrays	12
3.6	Controlling Conversion Path Display	12
3.7	Complex Compound Unit Conversions	13
3.8	Changing the Search Algorithm	13
4	The Units Network	14
4.1	Network Organization Principles	14
4.1.1	Canonical Definitions	14
4.1.2	No Duplicate Paths	14

4.1.3	Modular Structure	15
4.2	Length Network	15
4.2.1	SI Prefix Structure	15
4.2.2	Imperial Chain Structure	15
4.2.3	SI-Imperial Bridge	15
4.2.4	Complete Length Network with Highlighted Path	15
4.3	Time Network	16
4.4	Temperature Network	16
4.5	Other Network Segments	16
4.5.1	Volume Network	16
4.5.2	Pressure Network	17
4.5.3	Mass Network	17
4.6	Example Conversion Walkthroughs	17
4.6.1	Simple Conversion: Centimeters to Inches	17
4.6.2	Complex Conversion: stb/day/psi to m3/s/kPa	18
4.7	Summary	20
5	Under the Hood: The Directed Graph Foundation	21
5.1	Graph Theory Basics	21
5.1.1	What is a Directed Graph?	21
5.1.2	Graphs vs. Lookup Tables	22
5.2	unyts Implementation	22
5.2.1	The UNode Class	22
5.2.2	The Conversion Class	22
5.2.3	The UDigraph Class	23
5.2.4	Building the Network	23
5.3	Why Graphs Win	24
5.3.1	Automatic Path Discovery	24
5.3.2	Bidirectional Conversion	24
5.3.3	Composability	24
5.3.4	Extensibility	24
5.4	Dimensional Validation and Fail-Fast Mechanisms	25
5.4.1	The Problem: Invalid Dimensional Queries	25
5.4.2	The Solution: Empty Set Intersection	25
5.4.3	Dimensional Firewall Visualization	25
5.4.4	Bridging Disconnected Subgraphs	25
5.4.5	Error Handling as a Feature	26
5.5	Summary	27
6	Search Algorithms	28
6.1	Breadth-First Search (BFS)	28
6.1.1	Algorithm Overview	28
6.1.2	Implementation	28
6.1.3	Visual Example	29
6.1.4	Use Cases	29
6.2	Depth-First Search (DFS)	29
6.2.1	Algorithm Overview	29
6.2.2	Implementation with Descendant Filtering	30
6.2.3	Visual Example	30
6.2.4	Use Cases	30
6.3	lean_BFS (Optimized BFS)	30
6.3.1	Innovation: Network Pruning	30

6.3.2	Multi-Generation Strategy	31
6.3.3	Performance Gains	32
6.3.4	Use Cases	32
6.4	hybrid_BFS (Parallel Search)	32
6.4.1	Concurrent Execution	32
6.4.2	Parallelization Options	33
6.4.3	Use Cases	33
6.5	Algorithm Comparison	33
6.5.1	Selection Guidelines	33
6.6	Summary	34
7	Graphical User Interface	35
7.1	Main Window	35
7.2	Bidirectional Conversion	36
7.3	File Menu	36
7.3.1	Conversions Memory	36
7.3.2	Delete Cache Files	36
7.4	Options Menu	36
7.4.1	Set FVF (Formation Volume Factor)	36
7.4.2	Set Density	37
7.4.3	General Options	37
7.4.4	Search Algorithm	37
7.4.5	Parallel	38
7.5	Help Menu	38
7.6	Conversion Path Display	38
7.7	Error Handling in GUI	39
8	Python API Reference	40
8.1	Core Functions	40
8.1.1	convert()	40
8.1.2	units()	41
8.2	Unit Class	41
8.2.1	Attributes	42
8.2.2	Arithmetic Operations	42
8.2.3	Comparison Operators	43
8.2.4	.to() Method	43
8.3	Module-Level Functions	43
8.3.1	Configuration	43
8.3.2	Memory Management	44
8.3.3	Path Inspection	44
8.3.4	Custom Units and Conversions	44
8.3.5	Formation Volume Factor (FVF)	44
8.3.6	Constant Density	45
8.4	Type Checking and Subclasses	45
8.5	NumPy and Pandas Integration	45
8.6	Uncertain Units	46
9	Troubleshooting	47
9.1	Installation Issues	47
9.1.1	Module Not Found	47
9.1.2	Dependency Conflicts	47
9.2	Conversion Errors	47

9.2.1	NoConversionFoundError	47
9.2.2	SearchTimeoutError	48
9.2.3	NoFVLError	48
9.2.4	WrongFormatError	49
9.3	GUI Issues	49
9.3.1	GUI Won't Launch	49
9.3.2	GUI Crashes on Conversion	49
9.4	Performance Issues	49
9.4.1	Slow First Conversion	49
9.4.2	Slow Subsequent Conversions	50
9.5	Cache Corruption	50
9.6	Logging and Debugging	50
9.6.1	Enable Verbose Mode	50
9.6.2	Adjust Logging Level	50
9.6.3	Inspect Network Structure	51
9.7	Getting Help	51
A	Comprehensive Unit Reference	52
A.1	Length Units	52
A.2	Mass Units	52
A.3	Time Units	52
A.4	Temperature Units	52
A.5	Volume Units	52
A.6	Pressure Units	53
A.7	Pressure Gradient Units	53
A.8	Area Units	53
A.9	Density Units	53
A.10	Velocity / Speed Units	53
A.11	Rate Units	53
A.12	Force Units	53
A.13	Energy / Work / Heat Units	54
A.14	Power Units	54
A.15	Viscosity Units	54
A.16	Permeability Units	54
A.17	Data Units	54
A.18	Dimensionless Quantities	54
A.19	Notes on Unit Selection	54
A.19.1	SI vs. Imperial Choice	54
A.19.2	Petroleum Abbreviations	55
A.19.3	Temperature Notes	55
A.20	Extending the Unit Database	55
B	Foundational Conversion Definitions	56

Chapter 1

Introduction

unyts is an innovative Python package that revolutionizes unit conversion and unit-aware calculations by leveraging directed graph network theory combined with graph search algorithms.

1.1 The Core Innovation

Traditional unit converters rely on exhaustive lookup tables containing pre-computed conversion factors for every possible unit pair. This approach suffers from several limitations:

- Scalability: Adding new units requires defining conversions to all existing units
- Maintenance: Each new unit adds $O(n)$ conversion definitions
- Compound units: Ratios like **kg/m³** or **stb/day/psi** explode the lookup table size
- Memory overhead: Storing redundant conversion paths

The **unyt**s package takes a fundamentally different approach by modeling units and conversions as a directed graph (digraph):

- Each unit is a node in the graph
- Each conversion rule is a directed edge with a transformation function
- Breadth-First Search (BFS) discovers conversion paths automatically
- Units are defined by their official definitions (e.g., 1 foot = 12 inches)

This graph-based architecture enables conversion between any two compatible units without requiring explicit definition of every conversion pair. The search algorithms find the optimal path through the network automatically.

1.2 Key Features

1.2.1 Pre-Loaded Unit Networks

The package ships with comprehensive networks covering:

- Length: SI (meter with prefixes) and Imperial (inch, foot, yard, mile, etc.)
- Area: square units for all length systems

- Volume: cubic units, gallons (US & UK), liters, barrels
- Mass/Weight: metric (gram, kilogram) and Imperial (pound, ounce, ton)
- Time: millisecond through century
- Temperature: Celsius, Fahrenheit, Kelvin, Rankine
- Pressure: Pascal, bar, psi (gauge & absolute)
- Energy, Power, Density, Viscosity, Data, and more

1.2.2 Unit-Aware Calculations

The **Unit** class hierarchy provides:

- Arithmetic operations: Addition, subtraction, multiplication, division with automatic unit handling
- Logical comparisons: Units automatically convert before comparison
- Exponentiation: Raising units to powers (e.g., m^2 , ft^3)
- Compound unit creation: Multiplying **m** × **m** yields **m2**

1.2.3 Multiple Search Algorithms

Four specialized algorithms optimize different scenarios:

- BFS: Guarantees shortest path, layer-by-layer exploration
- lean_BFS: Optimized BFS with pre-filtered network (default)
- DFS: Fast depth-first search with descendant filtering
- hybrid_BFS: Parallel execution of BFS and lean_BFS

1.2.4 Dimensional Validation

The graph architecture also protects against invalid conversions. When a user requests a conversion between inherently incompatible dimensions (e.g., meters to kilograms without a density bridge), the search algorithms detect disconnected subgraphs and fail fast—returning an error in near-constant time rather than exhaustively exploring the entire network.

1.3 Use Cases

unyts excels in:

- Scientific computing: Heterogeneous unit handling in calculations
- Engineering applications: Converting between SI and Imperial systems
- Oil & gas industry: Specialized units (API gravity, standard barrels, formation volume factors)
- Data analysis: pandas/NumPy integration for unit-aware arrays
- Education: Teaching dimensional analysis and unit systems

1.4 Design Philosophy

1. Canonical definitions: Units defined by international standards (1 yard = 0.9144 meters)
2. No duplicate paths: Each unit pair connected by single canonical route
3. Composability: Complex ratios built from simple units
4. Extensibility: Add custom units without modifying existing network
5. Performance: Search memory caching for repeated conversions

The following chapters begin with installation and a quick-start guide so you can be up and running in minutes, then walk through the units network, the directed graph foundations, the search algorithms, the graphical user interface, and the full Python API reference.

Chapter 2

Installation

2.1 Requirements

2.1.1 Python Version

unyts supports Python 3.7 and later. The package metadata targets:

```
requires-python = ">=3.7"
```

Recommended: Python 3.10+ for optimal performance.

2.1.2 Core Dependencies

Required packages (automatically installed with **unyts**):

- **stringthings**: Helper utilities for string processing
- **cloudpickle**: Caching network and search memory

2.1.3 Optional Dependencies

While not required, the following packages enhance **unyts** functionality:

- NumPy: Enables array-aware conversions, handles lists/ndarrays
- Pandas: Recognizes Series/DataFrame objects for bulk conversions
- openpyxl: Required only for exporting the network to Excel format

Install optional dependencies manually if needed:

```
pip install numpy pandas openpyxl
```

2.2 Installation Methods

2.2.1 Install from PyPI (Recommended)

The simplest installation method:

```
pip install unyts
```

2.2.2 Upgrade to Latest Version

To ensure you have the latest features and bug fixes:

```
pip install --upgrade unyts
```

2.2.3 Install from Source

For development or testing unreleased features:

```
git clone https://github.com/ayaranitram/unyts.git
cd unyts
pip install -e .
```

The `-e` flag installs in editable mode, allowing you to modify source code and see changes immediately.

2.3 Verification

Verify the installation by checking the version:

```
1 import unyts
2 print(unyts.__version__) # Should print: 0.10.1 (or later)
```

Test basic functionality:

```
1 from unyts import convert
2
3 result = convert(1, 'meter', 'foot')
4 print(result) # Should print: 3.280839895013123
```

If no errors occur, `unyts` is correctly installed and operational.

2.4 First-Time Setup

On first import, `unyts` performs one-time initialization:

1. Builds the unit conversion network from definitions
2. Loads search memory cache (if available)
3. Creates user configuration directory

This process may take a few seconds. Subsequent imports are faster due to caching.

2.5 Configuration Files

`unyts` stores configuration and cache files in a user-specific directory:

- Windows: `C:\Users\<username>\.unyts\`
- macOS/Linux: `~/.unyts/`

Files stored:

- `parameters.ini`: User preferences (algorithm, print path, etc.)

- `search_memory.cache`: Cached conversion paths
- `units_network.cache`: Serialized network (future optimization)

To reset `unyts` to defaults, delete this directory and re-import.

2.6 Troubleshooting Installation

2.6.1 Missing cloudpickle

If you see warnings about cloudpickle:

```
Warning: Missing `cloudpickle` package.  
Not able to cache search memory.
```

Install it explicitly:

```
pip install cloudpickle
```

2.6.2 Import Errors

If imports fail with `ModuleNotFoundError`, ensure `unyts` is installed in the active Python environment:

```
pip list | grep unyts
```

If using virtual environments or conda, activate the correct environment before installation.

2.6.3 Version Conflicts

If upgrading causes issues, completely remove the old version first:

```
pip uninstall unyts  
pip install unyts
```

Chapter 3

Quick Start

This chapter provides a concise introduction to using **unyts** for common tasks.

3.1 Basic Conversion

The simplest way to use **unyts** is via the **convert()** function:

```
1 from unyts import convert
2
3 # Convert 10 feet to meters
4 meters = convert(10, 'ft', 'm')
5 print(meters) # Output: 3.048
```

The function signature is:

```
1 convert(value, from_units, to_units,
2         print_conversion_path=None, use_cache=True)
```

- **value**: Numeric input (int, float, list, np.array, pd.Series)
- **from_units**: Source unit string (e.g., 'ft', 'kg/m3')
- **to_units**: Target unit string
- **print_conversion_path**: Override global print_path setting
- **use_cache**: Set **False** to force fresh search

3.2 Creating Unit-Aware Quantities

Use the **units()** factory function to create **Unit** instances:

```
1 from unyts import units
2
3 q = units(6, 'in') + units(1, 'ft')
4 print(q) # Output: 18_in
```

Arithmetic operations automatically handle unit conversions:

```
1 length = units(5, 'm')
2 width = units(200, 'cm')
3 area = length * width
4 print(area) # Output: 10.0_m2
```

3.3 Type Checking

Check if a variable is unit-aware using `isinstance()`:

```
1 from unyts import Unit, units
2
3 value = units(5, 'm')
4 print(isinstance(value, Unit)) # Output: True
```

Important: Use `isinstance()`, not `type() is Unit`, because `units()` returns subclasses (e.g., `Length`, `Volume`).

3.4 Converting Unit Instances

Use the `.to()` method to convert instances:

```
1 length = units(100, 'cm')
2 print(length.to('in')) # Output: 39.37_in
3 print(length.to('ft')) # Output: 3.281_ft
```

3.5 Working with Arrays

`unyts` supports NumPy arrays and lists:

```
1 import numpy as np
2 from unyts import convert, units
3
4 # Convert list
5 temps_C = [0, 10, 20, 30]
6 temps_F = convert(temps_C, 'Celsius', 'Fahrenheit')
7 print(temps_F) # [32.0, 50.0, 68.0, 86.0]
8
9 # Unit-aware array
10 distances = units(np.array([1, 2, 3, 4, 5]), 'km')
11 print(distances.to('mi'))
12 # Output: [0.621 1.243 1.864 2.485 3.107]_mi
```

3.6 Controlling Conversion Path Display

By default, `unyts` prints the conversion path on first use:

```
1 from unyts import convert
2 convert(1, 'km', 'mi')
3 # Output: km > m > yard > foot > inch > ... > mile
4 #          0.6213711922373339
```

Toggle this behavior globally:

```
1 import unyts
2
3 unyts.print_path(False) # Disable
4 unyts.print_path(True)  # Enable
5 unyts.print_path()      # Toggle current state
```

Or control per-conversion:

```
1 convert(1, 'km', 'mi', print_conversion_path=True)
```

3.7 Complex Compound Unit Conversions

The true power of **unyts** shines with multi-dimensional compound units that would be impossible to enumerate in a lookup table. Here is a taste of what the graph architecture can do:

```
1 from unyts import convert
2
3 # Petroleum engineering: flow capacity in field units to SI
4 result = convert(500, 'stb/day/psi', 'm3/s/kPa')
5 print(result) # -> 1.334e-04
6
7 # The system automatically decomposes the compound unit,
8 # traverses 3 separate subgraphs (volume, time, pressure),
9 # and recombines the result -- no lookup table needed.
10
11 # Density: g/cm3 to lb/ft3
12 convert(2.65, 'g/cm3', 'lb/ft3') # -> 165.4
13
14 # Viscosity ratio
15 convert(1, 'cP/psi', 'Pa*s/kPa') # compound traversal
```

Chapter 4 (The Units Network) walks through exactly how the graph decomposes and solves the **stb/day/psi** conversion step by step.

3.8 Changing the Search Algorithm

Switch algorithms globally:

```
1 import unyts
2
3 unyts.set_algorithm('BFS')           # Standard BFS
4 unyts.set_algorithm('lean_BFS')     # Optimized (default)
5 unyts.set_algorithm('DFS')          # Depth-first
6 unyts.set_algorithm('hybrid_BFS')   # Parallel
```

See Chapter 6 for algorithm details and selection guidelines.

Chapter 4

The Units Network

This chapter explains how the unit conversion network is organized, how conversions are defined based on official standards, and visualizes key network segments.

4.1 Network Organization Principles

4.1.1 Canonical Definitions

Every unit conversion in **unyt**s is based on official international standards or widely accepted definitions:

- 1 foot = 12 inches (by definition)
- 1 yard = 3 feet (by definition)
- 1 yard = 0.9144 meters (exact, 1959 international yard)
- 1 kilometer = 1000 meters (SI prefix definition)
- 1 pound = 0.45359237 kilograms (exact, by definition)

This ensures consistency and prevents accumulation of rounding errors across multiple conversions.

4.1.2 No Duplicate Paths

A key design principle: between any two units, there exists exactly one canonical conversion path. This prevents:

- Conflicting conversion factors
- Cyclic dependencies with rounding errors
- Ambiguity in path selection

For example, to convert **foot** to **meter**, the path is uniquely:

foot → yard → meter

There is no direct **foot** → **meter** edge, ensuring all foot-to-meter conversions use the same computation.

4.1.3 Modular Structure

The network is organized into loosely-coupled modules:

1. SI System: Each base unit (meter, gram, second) connects to its SI prefixes
2. Imperial System: Units form chains based on their definitions
3. Bridges: Connect SI and Imperial systems at specific units
4. Specialized Systems: Temperature, pressure gauges, oil field units

4.2 Length Network

4.2.1 SI Prefix Structure

The meter serves as the SI base unit for length, with prefix multipliers defined by powers of 10:

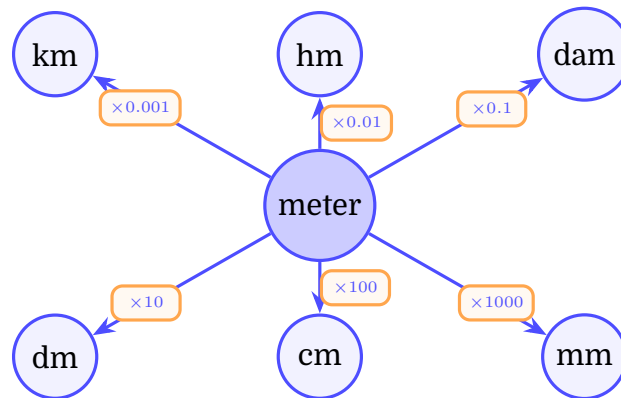


Figure 4.1: SI prefix network for length. Meter is the hub unit (blue center). Larger units above, smaller units below. All conversions route through the base unit. Circle nodes represent units, rectangular boxes represent conversion factors.

Additional SI prefixes (micro, nano, pico, kilo, mega, giga, etc.) follow the same pattern. The meter acts as a hub node for the SI subsystem.

4.2.2 Imperial Chain Structure

Imperial units form a linear chain based on their historical definitions:

4.2.3 SI-Imperial Bridge

The two systems connect at a single point: yard ↔ meter

4.2.4 Complete Length Network with Highlighted Path

Figure 4.4 shows the complete length network with a highlighted conversion path from centimeter to inch:

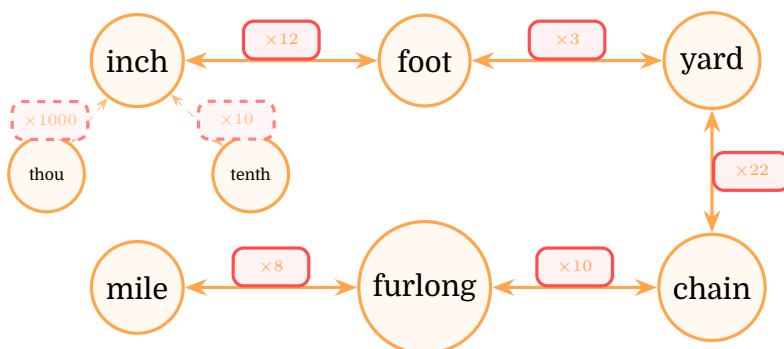


Figure 4.2: Imperial length chain (S-shaped layout). Top row flows left to right (inch→foot→yard), then wraps down to bottom row right to left (chain→furlong→mile). Circle nodes represent units, rectangular boxes represent conversion factors. Dashed lines show fractional sub-units.

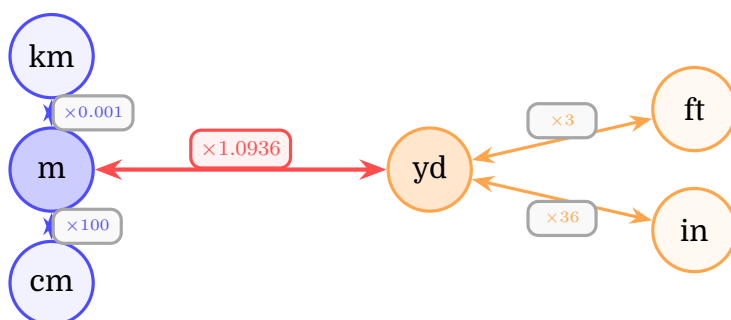


Figure 4.3: The SI-Imperial bridge. SI units (blue, left) connect to Imperial units (orange, right) through the meter↔yard bridge (red). All cross-system conversions must route through this connection. Circle nodes represent units, rectangular boxes represent conversion factors.

4.3 Time Network

The time network forms a hierarchical tree structure:

Note that **month** uses the average month length (30.4375 days) and conversions involving months are approximate. The year node connects to **day** (365.25 days) and **month** (12 months), providing both paths.

4.4 Temperature Network

Temperature conversions are non-linear due to offset + scale transformations:

The conversion functions for temperature include both multiplication and addition:

```
1 Celsius_to_Fahrenheit = lambda C: C * 9/5 + 32
2 Fahrenheit_to_Celsius = lambda F: (F - 32) * 5/9
```

Listing 4.1: Temperature conversion example

4.5 Other Network Segments

4.5.1 Volume Network

Volume conversions split into three subsystems:

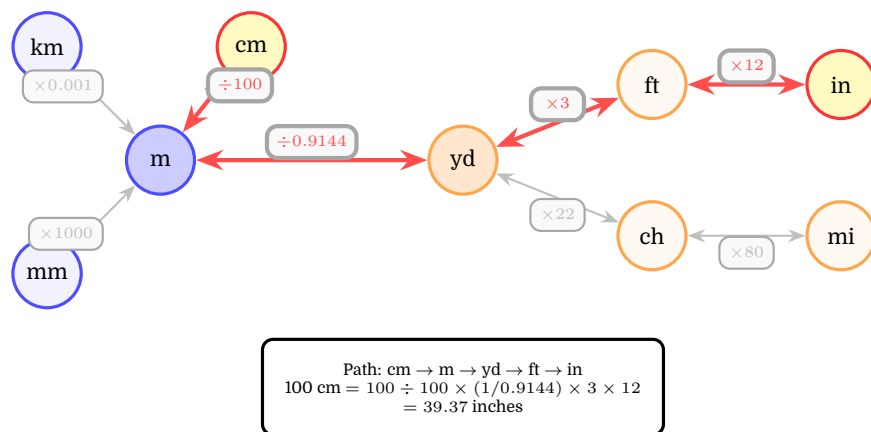


Figure 4.4: Complete length network showing highlighted conversion path from centimeter to inch (yellow nodes). Red thick arrows trace the conversion route. Gray connections show other available paths. Circle nodes represent units, rectangular boxes represent conversion factors.

- Cubic units: m^3 , cm^3 , ft^3 , in^3 (derived from length)
- Liquid volume (US): gallon $\rightarrow$ quart $\rightarrow$ pint $\rightarrow$ gill $\rightarrow$ fluid ounce
- Liquid volume (UK): Imperial gallon $\rightarrow$ quart $\rightarrow$ pint (different definitions than US)
- Bridge: Liter $\leftrightarrow \text{cm}^3$ (exact), gallon(UK) $\leftrightarrow$ liter, gallon(US) $\leftrightarrow$ cubic inch

4.5.2 Pressure Network

Pressure includes both absolute and gauge measurements:

- Absolute: Pascal (Pa), bar, psi(a), atmosphere, Torr
- Gauge: barg $\leftrightarrow$ bara (offset by atmospheric pressure), psig $\leftrightarrow$ psia
- Conversions: bar $\leftrightarrow$ Pascal $\leftrightarrow$ atmosphere, psi $\leftrightarrow$ bar

4.5.3 Mass Network

Similar to length, mass has SI (gram-based) and Imperial (pound-based) subsystems:

- SI: kilogram $\rightarrow$ gram (with prefixes: mg, kg, tonne)
- Imperial: pound $\rightarrow$ ounce $\rightarrow$ dram, ton $\rightarrow$ hundredweight $\rightarrow$ stone $\rightarrow$ quarter
- Bridge: pound $\leftrightarrow$ kilogram (exact conversion)

4.6 Example Conversion Walkthroughs

4.6.1 Simple Conversion: Centimeters to Inches

As a warm-up, consider converting 100 centimeters to inches. The search algorithm finds the path $\text{cm} \rightarrow \text{m} \rightarrow \text{yard} \rightarrow \text{foot} \rightarrow \text{inch}$, and applies:

$$100 \text{ cm} \div 100 \div 0.9144 \times 3 \times 12 = 39.37 \text{ inches}$$

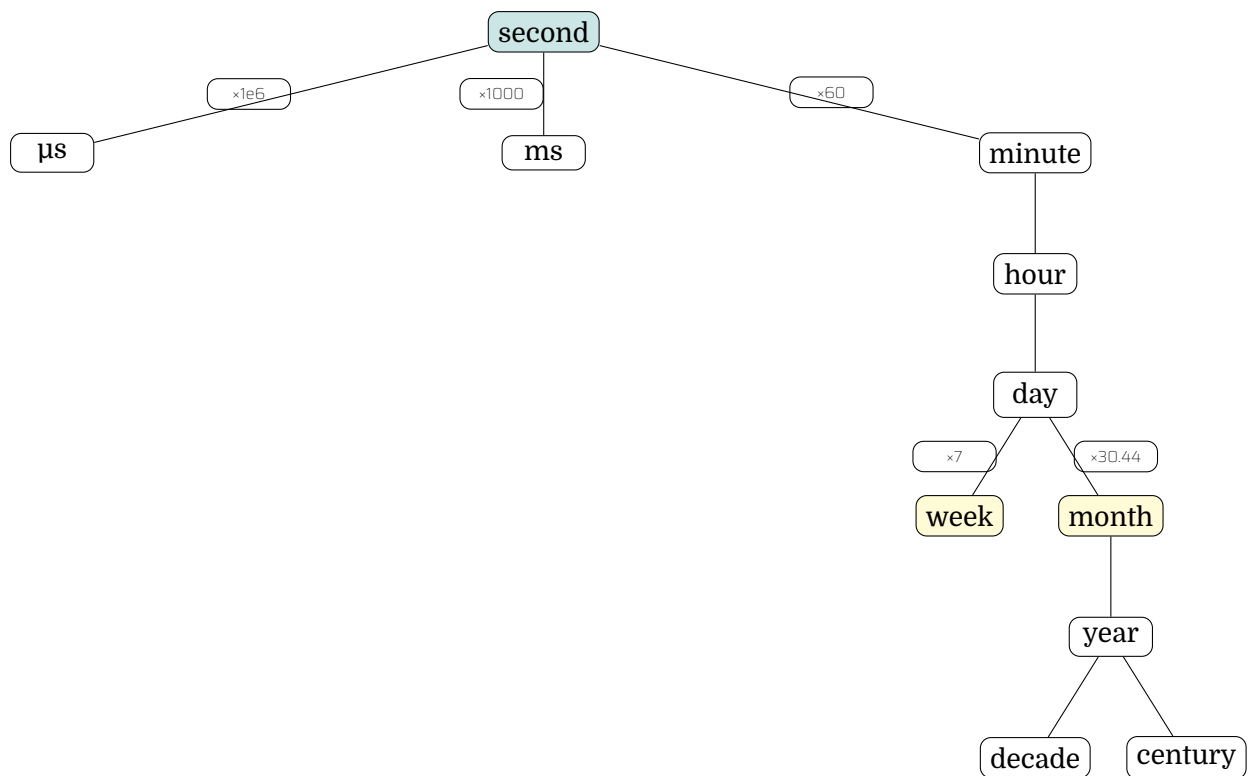


Figure 4.5: Time network hierarchical structure. Tree shows conversions from microseconds through seconds, minutes, hours, days to larger units. Yellow nodes (week, month) are alternatives branching from day.

This is a straightforward linear path. But a simple multiplication factor like $\times 0.3937$ could be hard-coded in any lookup table. To truly demonstrate the power of the graph architecture, we need a problem that no lookup table could practically solve.

4.6.2 Complex Conversion: stb/day/psi to $\text{m}^3/\text{s/kPa}$

In petroleum engineering, a common flow-capacity unit is standard barrels per day per psi (stb/day/psi). Converting this to SI units ($\text{m}^3/\text{s/kPa}$) is a multi-dimensional problem that requires simultaneously traversing three separate subgraphs: volume, time, and pressure.

A lookup table approach would need to enumerate every possible combination of volume, time, and pressure unit pairs—easily millions of rows. The graph handles it compositionally.

Step 1: Decompose the Compound Unit

The converter recognizes the ratio structure and decomposes it:

$$\frac{\text{stb}}{\text{day} \cdot \text{psi}} \rightarrow \frac{\text{m}^3}{\text{s} \cdot \text{kPa}}$$

This creates three independent subproblems:

1. Volume: $\text{stb} \rightarrow \text{m}^3$ (numerator)
2. Time: $\text{day} \rightarrow \text{s}$ (first denominator)
3. Pressure: $\text{psi} \rightarrow \text{kPa}$ (second denominator)

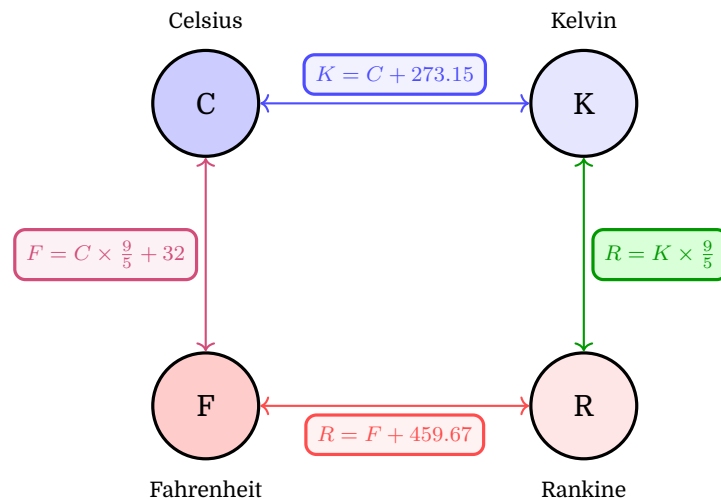


Figure 4.6: Temperature network showing all bidirectional conversions in rectangular boxes. Each conversion encodes both offset and scale transformations. Circle nodes represent temperature scales, rectangular boxes show conversion formulas.

Step 2: Traverse Three Subgraphs

Each subproblem is solved independently via graph search:

Volume subgraph:

`stb → bbl → gallon(US) → in3 → cm3 → liter → m3`

$$\begin{aligned}
 1 \text{ stb} &= 1 \text{ bbl} = 42 \text{ gal(US)} = 42 \times 231 \text{ in}^3 \\
 &= 9702 \text{ in}^3 \times 16.387 = 158\,987 \text{ cm}^3 \\
 &= 158.987 \text{ liters} = 0.158987 \text{ m}^3
 \end{aligned}$$

Time subgraph:

`day → hour → minute → second`

$$1 \text{ day} = 24 \times 60 \times 60 = 86\,400 \text{ seconds}$$

Pressure subgraph:

`psi → Pa → kPa`

$$1 \text{ psi} = 6894.76 \text{ Pa} = 6.89476 \text{ kPa}$$

Step 3: Recombine

The final conversion factor is assembled from the three independent results:

$$1 \frac{\text{stb}}{\text{day} \cdot \text{psi}} = \frac{0.158987 \text{ m}^3}{86\,400 \text{ s} \times 6.89476 \text{ kPa}} = 2.668 \times 10^{-7} \frac{\text{m}^3}{\text{s} \cdot \text{kPa}}$$

Step 4: Cache the Composite Result

The entire compound conversion factor (2.668×10^{-7}) is cached. The next time any code requests this exact conversion, it is retrieved in $O(1)$ constant time—no graph traversal needed:

```
1 from unyts import convert
2
3 # First call: traverses 3 subgraphs (volume, time, pressure)
4 result = convert(500, 'stb/day/psi', 'm3/s/kPa')
5 # -> 1.334e-04
6
7 # Second call: instant cached retrieval
8 result2 = convert(1000, 'stb/day/psi', 'm3/s/kPa')
9 # -> 2.668e-04 (no search needed)
```

This multi-dimensional decomposition is the core reason the graph architecture exists. No lookup table could practically pre-enumerate every combination of volume, time, and pressure unit conversions. The graph discovers the path automatically, caches it, and handles any new compound ratio on the fly.

Adding the Formation Volume Factor

In reservoir engineering, surface volumes (stb) and reservoir volumes (rb) differ by the Formation Volume Factor (FVF). Setting the FVF adds a bridge edge between the surface-volume and reservoir-volume subgraphs, enabling even more complex conversions:

```
1 import unyts
2
3 # Set FVF (reservoir volume / surface volume)
4 unyts.set_fvf(1.25)
5
6 # Now converts through the FVF bridge:
7 # stb -> rb (via FVF) -> ... -> rm3
8 result = convert(1000, 'stb/day', 'rb/day')
9 # -> 1250.0 rb/day (1000 * 1.25)
```

The FVF dynamically adds an edge to the graph. Once set, the system automatically discovers and uses this bridge whenever a conversion path requires it.

4.7 Summary

The **unyts** network is carefully organized around:

1. Official definitions: All conversions based on international standards
2. Single canonical paths: No duplicate routes between unit pairs
3. Modular structure: SI, Imperial, and specialized subsystems
4. Strategic bridges: Minimal connections between subsystems
5. Hub topology: Base units (meter, gram, second) serve as central nodes

This architecture enables automatic discovery of conversion paths for thousands of unit combinations from a compact set of defining relationships.

Chapter 5

Under the Hood: The Directed Graph Foundation

This chapter explains the graph-theoretic foundations of **unyts**, demonstrating why directed graphs provide superior unit conversion compared to traditional lookup tables.

5.1 Graph Theory Basics

5.1.1 What is a Directed Graph?

A directed graph (digraph) $G = (V, E)$ consists of:

- A set of vertices (nodes) $V = \{v_1, v_2, \dots, v_n\}$
- A set of directed edges $E \subseteq V \times V$
- Each edge $e = (v_i, v_j)$ points from source v_i to target v_j

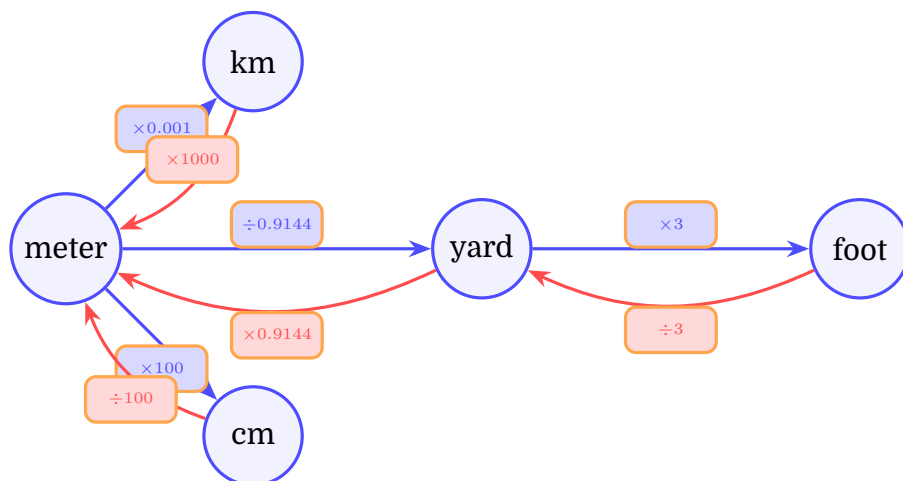


Figure 5.1: Simple directed graph showing length unit conversions. Circle nodes represent units. Rectangular boxes represent conversion factors. Blue edges represent forward conversions, red edges represent reverse conversions.

In Figure 5.1, each node represents a unit, and directed edges encode conversion factors. Notice that edges are bidirectional—both forward and reverse conversions exist.

5.1.2 Graphs vs. Lookup Tables

Feature	Lookup Table	Graph Network
Add 1 unit	Define n conversions	Define 1–2 edges
Storage	$O(n^2)$	$O(n)$ or $O(n \log n)$
Query time	$O(1)$	$O(n)$ with caching $\rightarrow O(1)$
Compound units	Exponential explosion	Compositional
Maintenance	Duplicate definitions	Canonical paths
Extensibility	Rigid	Dynamic

Table 5.1: Comparison of lookup tables vs. graph-based conversion

The key advantage: adding a new unit to a graph requires connecting it to one or two existing units (e.g., connecting a new length unit only to **meter**), whereas a lookup table requires defining conversions to all existing units.

5.2 unyts Implementation

5.2.1 The UNode Class

Each unit in the network is represented by a **UNode** object:

```

1 class UNode:
2     __slots__ = ('name',)
3
4     def __init__(self, name: str):
5         self.name = name
6
7     def __eq__(self, other):
8         return self.name == other.name
9
10    def __hash__(self):
11        return hash(self.name)

```

Listing 5.1: UNode implementation (simplified)

Key design choices:

- Equality based on name: Nodes created in different sessions compare equal
- Hashable: Enables use as dictionary keys for search memory caching
- Lightweight: Minimal memory footprint with `__slots__`

5.2.2 The Conversion Class

Directed edges encode transformation functions:

```

1 class Conversion:
2     def __init__(self, source: UNode, target: UNode,
3                 function: callable):
4         self.source = source
5         self.target = target
6         self.function = function

```

Listing 5.2: Conversion class structure

A conversion from meters to kilometers stores:

```

1 meter_to_km = Conversion(
2     source=UNode('meter'),
3     target=UNode('kilometer'),
4     function=lambda x: x * 0.001
5 )

```

5.2.3 The UDigraph Class

The master graph object manages the entire network:

```

1 class UDigraph:
2     def __init__(self):
3         self.edges = {} # {UNode: ([children], [conversions])}
4         self.memory = {} # Cached search results
5         self.fvf = None # Formation volume factor
6
7     def add_node(self, node: UNode):
8         if node not in self.edges:
9             self.edges[node] = ([], [])
10
11    def add_edge(self, conversion: Conversion):
12        source = conversion.source
13        if source not in self.edges:
14            self.add_node(source)
15        children, conversions = self.edges[source]
16        children.append(conversion.target)
17        conversions.append(conversion.function)
18
19    def children_of(self, node: UNode):
20        return self.edges[node][0] if node in self.edges else []

```

Listing 5.3: UDigraph core attributes

The `edges` dictionary maps each node to a tuple of (`child_nodes`, `conversion_functions`), enabling efficient traversal during search.

5.2.4 Building the Network

Unit conversions are defined based on official standards:

```

1 network = UDigraph()
2
3 # Define nodes
4 meter = UNode('meter')
5 kilometer = UNode('kilometer')
6 yard = UNode('yard')
7 foot = UNode('foot')
8
9 network.add_node(meter)
10 network.add_node(kilometer)
11 network.add_node(yard)
12 network.add_node(foot)
13
14 # Define conversions (both directions)
15 network.add_edge(Conversion(meter, kilometer,
16                             lambda x: x * 0.001))

```



```

17 network.add_edge(Conversion(kilometer, meter,
18                             lambda x: x * 1000))
19 network.add_edge(Conversion(meter, yard,
20                             lambda x: x / 0.9144))
21 network.add_edge(Conversion(yard, meter,
22                             lambda x: x * 0.9144))
23 network.add_edge(Conversion(yard, foot,
24                             lambda x: x * 3))
25 network.add_edge(Conversion(foot, yard,
26                             lambda x: x / 3))

```

Listing 5.4: Example network construction

5.3 Why Graphs Win

5.3.1 Automatic Path Discovery

Once the network is built, conversions between any compatible unit pair are automatic. To convert 100 centimeters to feet:

1. BFS finds path: **cm** → **m** → **yard** → **foot**
2. Apply transformations: $100 \div 100 \times (1/0.9144) \times 3 = 3.281$ feet
3. Cache result for future use

No explicit **cm** → **foot** conversion was ever defined!

5.3.2 Bidirectional Conversion

Every conversion automatically works in both directions. Defining **meter** → **yard** and its inverse enables conversions in either direction plus all transitive paths (**meter** → **yard** → **foot**).

5.3.3 Composability

Compound units emerge naturally:

- **kg/m3** decomposes to **kg** and **m3**
- Each component converts independently
- **stb/day/psi** = (standard barrels) / (day) / (pounds per square inch)

The graph handles numerator and denominator units separately, then recombines.

5.3.4 Extensibility

Adding industry-specific units requires minimal changes:

```

1 from unyts import set_unit, set_conversion
2
3 # Add a custom unit
4 set_unit('my_unit')
5
6 # Define conversion to an existing unit
7 set_conversion('my_unit', 'meter',

```

```

8         lambda x: x * 42.0,
9         lambda x: x / 42.0)

```

Listing 5.5: Adding custom units

The new unit immediately participates in all graph search operations, enabling conversions to any compatible unit without additional definitions.

5.4 Dimensional Validation and Fail-Fast Mechanisms

Good documentation doesn't just show how software behaves when everything goes right—it reassures the user that the software won't crash their system when things go wrong. This section explains how **unyt**s efficiently handles impossible conversion requests.

5.4.1 The Problem: Invalid Dimensional Queries

In real-world engineering and data science workflows, users frequently make typos or mistakenly request conversions between fundamentally incompatible dimensions—for example, converting meters to kilograms without setting a density bridge. In a large graph network with hundreds of unit nodes, a naive search algorithm could waste significant compute cycles spinning through a completely disconnected subgraph, looking for a path that simply does not exist.

5.4.2 The Solution: Empty Set Intersection

The **lean_BFS** algorithm's descendant intersection strategy provides an elegant fail-fast mechanism. Recall that **lean_BFS** works by:

1. Computing the descendants of the start node: D_{start}
2. Computing the descendants of the end node: D_{end}
3. Finding the intersection: $D_{relevant} = D_{start} \cap D_{end}$

When the start and end nodes belong to disconnected subgraphs (e.g., the mass subgraph and the length subgraph), their descendant sets have no common members:

$$D_{\text{kilogram}} \cap D_{\text{meter}} = \emptyset$$

This empty intersection is detected immediately at the very first generation depth, triggering termination in near-constant time $O(1)$ —regardless of the total size of the network. The algorithm simply stops rather than uselessly exhausting all 25 generation depths.

5.4.3 Dimensional Firewall Visualization

Figure 5.2 illustrates how disconnected subgraphs act as a dimensional firewall, preventing wasteful search across incompatible unit categories.

5.4.4 Bridging Disconnected Subgraphs

The user can explicitly bridge otherwise disconnected dimensions by setting physical constants:

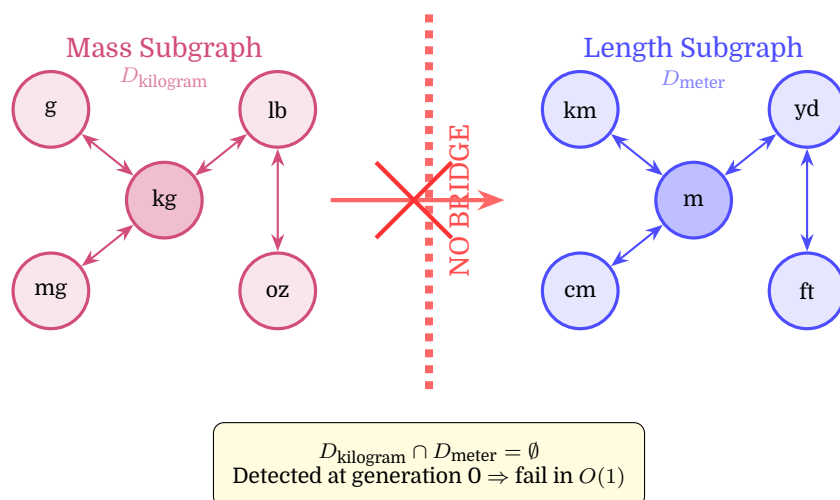


Figure 5.2: Dimensional firewall between mass and length subgraphs. Without a density bridge connecting these two disconnected components, the `lean_BFS` intersection immediately produces an empty set, halting the search in constant time. The red dashed line and crossed-out arrow represent the absence of any connecting path.

```

1 import unyts
2
3 # Without density: fails fast with NoConversionFoundError
4 # convert(1, 'liter', 'kg') # -> error
5
6 # Set density bridge
7 unyts.set_density(997) # g/cm3 (water at 25 C)
8
9 # Now works: volume <-> mass via density
10 result = unyts.convert(1, 'liter', 'kg') # -> 0.997 kg

```

Listing 5.6: Bridging mass and volume with density

Setting the density effectively adds a temporary edge connecting the volume subgraph to the mass subgraph, enabling the search to traverse between these previously disconnected worlds.

5.4.5 Error Handling as a Feature

The fail-fast behavior is not merely an error code—it is a computationally defensive feature that:

1. Protects CPU: Prevents wasteful exhaustive searches through the entire network
2. Protects memory: No exponential queue growth from expanding disconnected branches
3. Provides fast feedback: Users learn immediately that a density or FVF bridge is needed, rather than waiting for a timeout
4. Scales gracefully: The $O(1)$ detection time holds regardless of whether the network has 100 or 10,000 nodes

5.5 Summary

The directed graph architecture provides:

1. Scalability: $O(n)$ storage vs. $O(n^2)$ lookup tables
2. Maintainability: Canonical definitions prevent inconsistencies
3. Flexibility: Automatic discovery of conversion paths
4. Composability: Natural handling of compound units
5. Extensibility: Add units without modifying existing network

The next chapter explores the graph search algorithms that traverse this network efficiently. Chapter 4 (The Units Network) provides concrete examples of how this architecture is applied in practice.

Chapter 6

Search Algorithms

unytS implements four specialized graph search algorithms, each optimized for different conversion scenarios. This chapter explains their operation, performance characteristics, and use cases.

6.1 Breadth-First Search (BFS)

6.1.1 Algorithm Overview

BFS explores the graph layer by layer, visiting all nodes at distance d before exploring nodes at distance $d + 1$. This guarantees finding the shortest path between two nodes.

Key properties:

- Completeness: Finds a path if one exists
- Optimality: Always returns the shortest path
- Time complexity: $O(|V| + |E|)$ where V = nodes, E = edges
- Space complexity: $O(|V|)$ for the queue and visited set

6.1.2 Implementation

```
1 def BFS(graph, start, end):
2     path_queue = [[start]]      # Queue of paths
3     visited = []                # Visited paths
4
5     while path_queue:
6         current_path = path_queue.pop(0) # Dequeue
7         last_node = current_path[-1]
8
9         if current_path in visited:
10             continue
11         visited.append(current_path)
12
13         if last_node == end:
14             return current_path # Found!
15
16         # Expand to children
17         for child in graph.children_of(last_node):
18             if child not in current_path:
19                 new_path = current_path + [child]
```

```

20     path_queue.append(new_path)
21
22     return None # No path found

```

Listing 6.1: BFS pseudocode

6.1.3 Visual Example

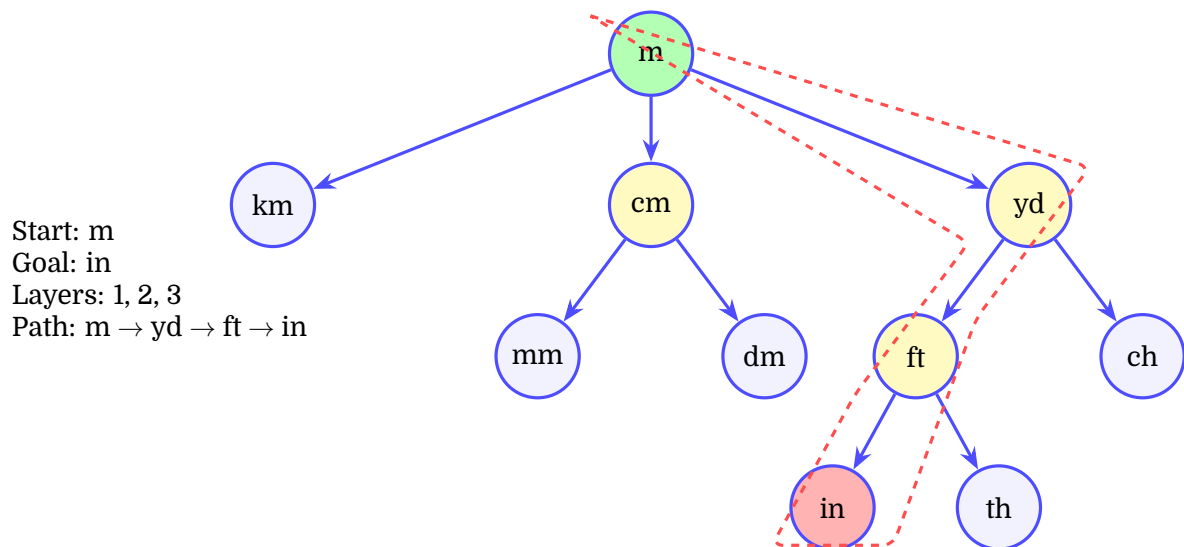


Figure 6.1: BFS exploration tree. Green node is start, yellow nodes explored before red goal node (in). Dashed red line shows shortest path found. All unit nodes are circles.

In Figure 6.1, BFS explores level 1 (km, cm, yd), then level 2 (mm, dm, ft, ch), discovering the shortest 3-hop path: meter → yard → foot → inch.

6.1.4 Use Cases

- When shortest conversion path is required
- Unknown network topology
- Verification of conversion accuracy
- Teaching/debugging conversion logic

6.2 Depth-First Search (DFS)

6.2.1 Algorithm Overview

DFS explores as deep as possible along each branch before backtracking. It uses descendant filtering to avoid exploring branches that cannot reach the goal.

Key properties:

- Completeness: Finds a path (but not necessarily shortest)
- Optimality: No guarantee of shortest path
- Time complexity: $O(|V| + |E|)$ with pruning

- Space complexity: $O(|V|)$ for recursion stack
- Advantage: Often faster for deep paths

6.2.2 Implementation with Descendant Filtering

The `unyt`s DFS implementation adds intelligent pruning:

```

1 def DFS(graph, start, end, branch_depth=25):
2     def dfs_recursive(node, visited, path):
3         visited.add(node)
4
5         for child in graph.children_of(node):
6             if child in visited:
7                 continue
8
9             # Pruning: skip if goal unreachable within depth
10            if end not in get_descendants(child, branch_depth):
11                continue
12
13            new_path = path + [child]
14
15            if child == end:
16                return new_path # Found!
17
18            result = dfs_recursive(child, visited, new_path)
19            if end in result:
20                return result
21
22        return path
23
24    return dfs_recursive(start, set(), [start])

```

Listing 6.2: DFS with descendant filtering

The `get_descendants(node, depth)` function precomputes all nodes reachable within `depth` hops, preventing exploration of dead-end branches.

6.2.3 Visual Example

DFS may explore fewer nodes than BFS if the goal happens to be on the first deep branch explored (though it might also explore more if unlucky).

6.2.4 Use Cases

- Known deep conversion paths (e.g., complex compound units)
- Memory-constrained environments
- When first valid path is acceptable

6.3 lean_BFS (Optimized BFS)

6.3.1 Innovation: Network Pruning

The classic BFS bottleneck is exploring nodes irrelevant to the start→end conversion. `lean_BFS` addresses this by:

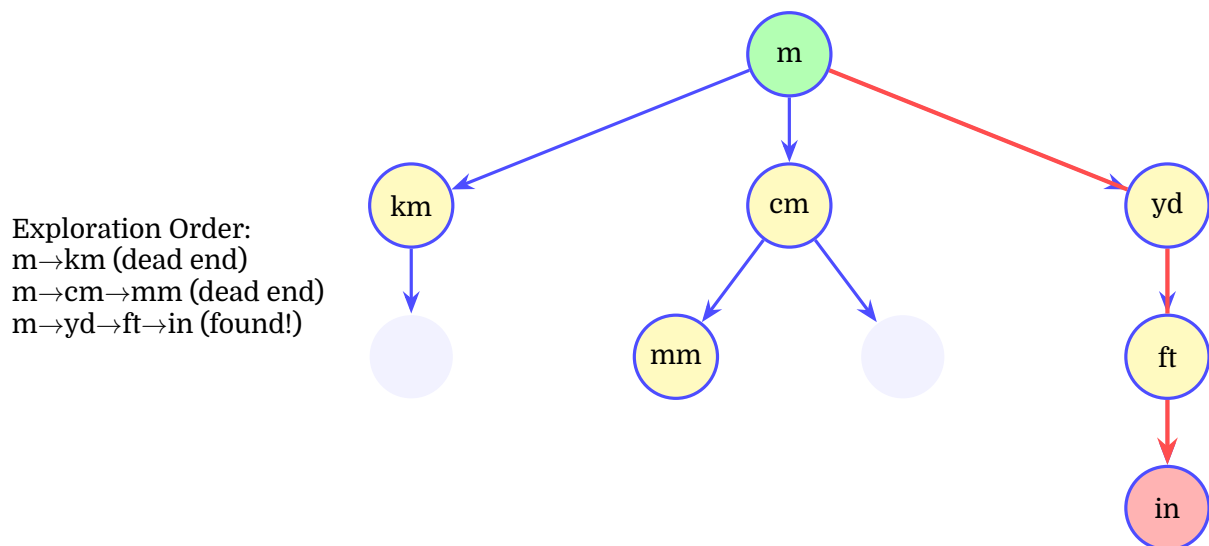


Figure 6.2: DFS exploration tree. Thick red arrows show the path DFS follows. Yellow nodes explored before finding goal. All unit nodes are circles.

1. Compute descendants of **start** up to depth d : D_{start}
2. Compute descendants of **end** up to depth d : D_{end}
3. Find intersection: $D_{relevant} = D_{start} \cap D_{end}$
4. Construct slimmed network containing only nodes in $D_{relevant}$
5. Run standard BFS on this smaller network

6.3.2 Multi-Generation Strategy

lean_BFS iteratively tries increasing depths: $[0, 1, 2, 3, 4, 5, 7, 10, 13, 16, 20, 25, \dots]$ until the intersection is non-empty. This avoids over-expanding for simple conversions while handling complex ones.

```

1 def lean_BFS(graph, start, end, max_gen=25):
2     generations = [0, 1, 2, 3, 4, 5, 7, 10, 13, 16, 20, 25]
3
4     for depth in generations:
5         if depth > max_gen:
6             break
7
8         start_desc = get_descendants(start, depth)
9         end_desc = get_descendants(end, depth)
10        intersection = start_desc & end_desc
11
12        if intersection:
13            # Build slim network with only these nodes
14            slim_edges = {
15                node: graph.edges[node]
16                for node in intersection
17                if node in graph.edges
18            }
19            slim_graph = SlimUDigraph(slim_edges)
20            return BFS(slim_graph, start, end)
21

```



```
22     return None # No path found
```

Listing 6.3: lean_BFS network filtering

6.3.3 Performance Gains

Typical performance improvement over standard BFS:

- Network size reduction: From 500+ nodes to 10-30 relevant nodes
- Speed improvement: 5-20× faster for typical conversions
- Memory savings: Proportional to network reduction

6.3.4 Use Cases

- Default algorithm for most conversions
- Production systems requiring fast response
- Repeated conversions between popular unit pairs

6.4 hybrid_BFS (Parallel Search)

6.4.1 Concurrent Execution

hybrid_BFS runs BFS and lean_BFS simultaneously in separate threads (or processes), returning the first successful result:

```
1 def hybrid_BFS(graph, start, end):
2     results = {'bfs': '', 'lean_bfs': ''}
3
4     thread_bfs = Thread(target=run_bfs,
5                         args=(results, graph, start, end))
6     thread_lean = Thread(target=run_lean_bfs,
7                          args=(results, graph, start, end))
8
9     thread_bfs.start()
10    thread_lean.start()
11
12    while True:
13        if results['lean_bfs']:
14            thread_bfs.terminate()
15            return results['lean_bfs']
16        elif results['bfs']:
17            thread_lean.terminate()
18            return results['bfs']
19        elif results['bfs'] is None and results['lean_bfs'] is None:
20            return None # Both failed
```

Listing 6.4: hybrid_BFS parallel execution

6.4.2 Parallelization Options

`unyts` supports multiple parallel backends:

- Threading: Default for Python <3.13 (GIL-limited)
- Multiprocessing: Python 3.14+ with free-threading (true parallelism)
- Auto: Automatically selects best available method

6.4.3 Use Cases

- Unknown path complexity
- Time-critical conversions
- Multi-core systems
- When willing to trade CPU for speed

6.5 Algorithm Comparison

Algorithm	Shortest Path	Speed	Memory	Best For
BFS	Yes	Medium	High	Shortest guarantee
DFS	No	Fast (deep)	Low	Known deep paths
lean_BFS	Yes	Fast	Medium	Most conversions
hybrid_BFS	Yes	Fastest	Highest	Critical speed

Table 6.1: Search algorithm comparison

6.5.1 Selection Guidelines

- Default: Use `lean_BFS` (fast + optimal)
- Verification: Use `BFS` to confirm shortest path
- Complex units: Try `DFS` for deep compound ratios
- Maximum speed: Use `hybrid_BFS` with multiprocessing

The algorithm can be changed globally or per-conversion:

```
1 import unyts
2
3 # Set globally
4 unyts.set_algorithm('hybrid_BFS')
5
6 # All conversions use hybrid_BFS from now on
7 result = convert(100, 'cm', 'in')
```

6.6 Summary

unytS provides four specialized search algorithms, each optimized for different scenarios. The default **lean_BFS** combines the shortest-path guarantee of BFS with network pruning for excellent performance. Users can select algorithms based on their specific needs: verification (BFS), deep paths (DFS), or maximum speed (hybrid_BFS).

Chapter 7

Graphical User Interface

The Unyts GUI provides an intuitive visual interface for unit conversions. Launch it from Python:

```
1 import unyts
2 unyts.start_gui()
```

Or from the command line:

```
1 python -m unyts
```

7.1 Main Window

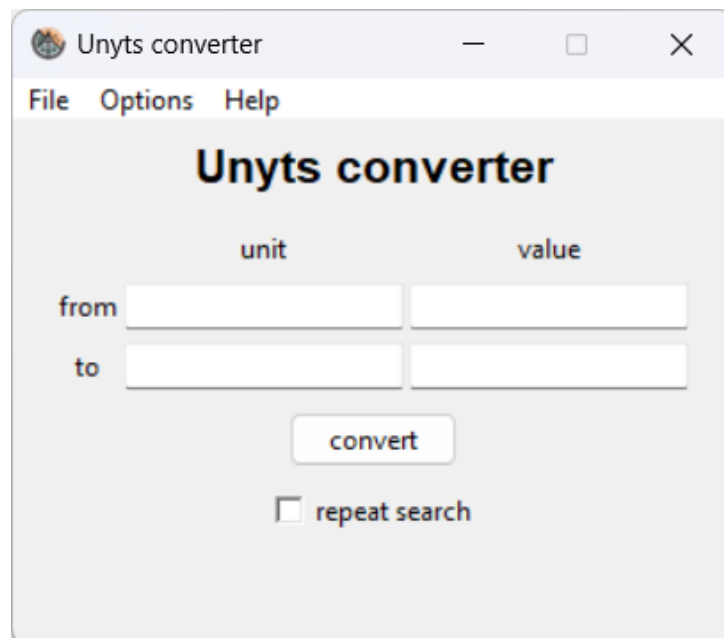


Figure 7.1: Unyts GUI main window showing conversion interface

The main window (Figure 7.1) features:

- From unit and From value: Source unit and quantity
- To unit and To value: Target unit and result

- Convert button: Execute conversion (or press Enter in any field)
- Repeat search checkbox: Force re-search (ignores cache)
- Conversion path display: Shows graph traversal path when enabled

7.2 Bidirectional Conversion

The GUI supports conversions in both directions:

- Forward: Enter value and unit in "from" fields, specify target "to" unit → press Enter or click Convert
- Reverse: Enter desired value in "to" field → calculates required "from" value

This bidirectional flexibility eliminates the need to invert calculations manually.

7.3 File Menu

7.3.1 Conversions Memory

The memory subsystem caches conversion factors to accelerate repeated conversions:

- Save memory: Persist cache to disk (happens automatically on exit if **Use cache** enabled)
- Clean memory: Clear all cached conversion factors from RAM
- Reload memory: Refresh cache from disk file
- Export memory: Save cache to custom file location
- Import memory: Load cache from external file

Use export/import to share conversion caches between machines or backup expensive search results.

7.3.2 Delete Cache Files

Removes all **.cache** files from the Unyts user directory. Useful for troubleshooting or forcing complete rebuild.

7.4 Options Menu

7.4.1 Set FVF (Formation Volume Factor)

For petroleum engineering workflows converting between reservoir and surface volumes. Enter FVF in:

- Dimensionless form: **1.25 vol/vol**
- With units: **1.5 rb/MMscf**

Once set, enables conversions like `convert(10, 'rb', 'sm3')` using FVF correction. For example, to execute the **stb/day/psi** compound conversion discussed in Chapter 4, a petroleum engineer would first set the FVF here, then perform the conversion—the GUI displays the full graph traversal path in the marquee below the Convert button.

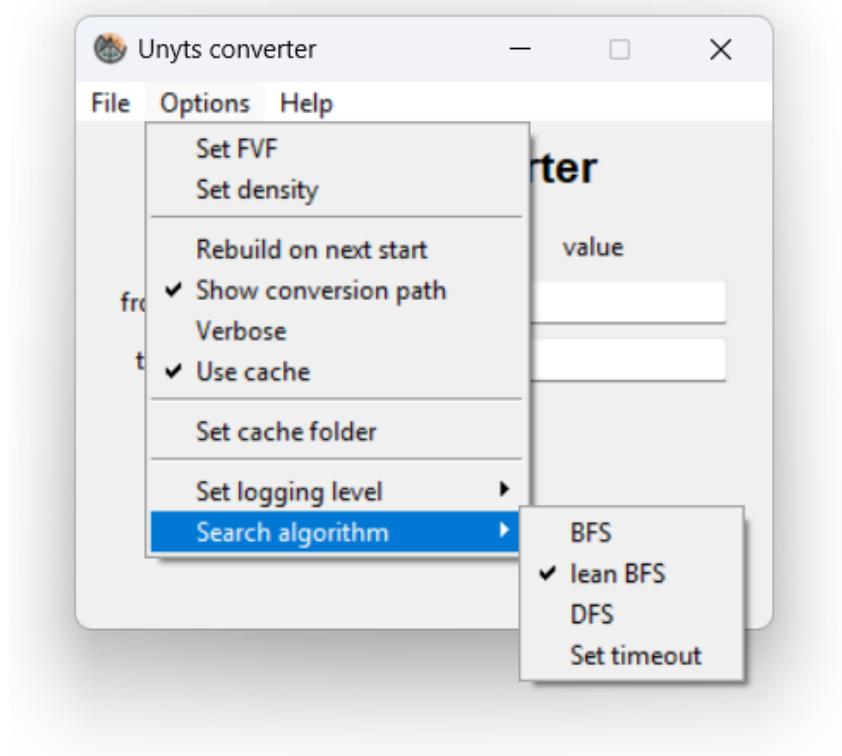


Figure 7.2: Options menu showing search algorithm selection

7.4.2 Set Density

Constant density for volume ↔ mass conversions. Enter in:

- Default: **997 g/cm³**
- With units: **8.34 lb/gal**

Enables conversions between incompatible dimensions, e.g., `convert(1, 'liter', 'kg')` using density bridge.

7.4.3 General Options

- Rebuild on next start: Force full network rebuild from database definitions (clears cache)
- Show conversion path: Display graph traversal path in marquee below button
- Verbose: Enable detailed console logging
- Use cache: Persist conversion factors across sessions
- Set cache folder: Change location where **.cache** files are stored
- Set logging level: Choose DEBUG, INFO, WARNING, ERROR, or CRITICAL verbosity

7.4.4 Search Algorithm

(Figure 7.2)

Select graph search algorithm:

- BFS: Standard breadth-first (guaranteed shortest path)
- lean BFS: Optimized BFS (default, 2.5× faster)
- hybrid BFS: Parallel BFS (experimental, requires threading/multiprocessing)
- DFS: Depth-first (faster for dense graphs, no guarantee of shortest path)
- Set timeout: Maximum search time in seconds (default 5, enter **unlimited** for no limit)

Changing algorithm automatically selects "Repeat search" to apply the new method.

7.4.5 Parallel

Enable experimental parallel search (requires Python threading or multiprocessing support):

- threading: Use Python threads (lighter weight)
- multiprocessing: Use separate processes (better CPU utilization)
- off: Disable parallelization
- auto: Automatic selection based on platform capabilities

Note: Parallel modes are currently experimental and deactivated by default on some platforms.

7.5 Help Menu

- Help: Opens Jupyter notebook demo at https://github.com/ayaranitram/unyts/blob/master/unyts_demo.ipynb
- GitHub: Links to repository at <https://github.com/ayaranitram/unyts>
- Version: Displays current Unyts version number

7.6 Conversion Path Display

When "Show conversion path" is enabled, a scrolling marquee below the button shows:

- Graph traversal path with conversion factors
- Error messages (e.g., "no conversion found", "search timeout")
- Status updates (e.g., "FVF set as 1.25 vol/vol")

Example path display:

```
[Unyts INFO]: converting from 'cm' to 'in': cm ×0.3937→ in
```

7.7 Error Handling in GUI

The GUI gracefully handles:

- `NoConversionFoundError`: Displays "no conversion found" in path marquee
- `SearchTimeoutError`: Shows "search timeout!" message
- `NoFVFError`: Prompts user to "Please set FVF constant from Options menu"
- Invalid input: Non-numeric values are ignored until valid entry

Chapter 8

Python API Reference

The Unyts Python API provides functional and object-oriented interfaces for unit conversion, arithmetic, and manipulation.

8.1 Core Functions

8.1.1 `convert()`

Primary conversion function.

Signature:

```
1 convert(value, from_unit, to_unit, algorithm='lean_BFS',  
2         parallel=None, timeout=5, verbose=False)
```

Parameters:

- **value**: Numeric (int, float, array) to convert
- **from_unit**: Source unit as string (e.g., 'cm', 'kg/m3')
- **to_unit**: Target unit as string
- **algorithm**: 'BFS', 'lean_BFS' (default), 'DFS', 'hybrid_BFS'
- **parallel**: None (auto), 't' (threading), 'p' (multiprocessing), False
- **timeout**: Maximum search time seconds (default 5)
- **verbose**: Boolean to enable logging

Returns: Converted numeric value

Raises:

- **NoConversionFoundError**: No path exists in graph
- **SearchTimeoutError**: Search exceeded timeout
- **NoFVFError**: FVF required but not set
- **WrongFormatError**: Invalid unit string format

Examples:

```

1 from unyts import convert
2
3 # Basic conversion
4 convert(100, 'cm', 'm') # → 1.0
5
6 # With NumPy array
7 import numpy as np
8 convert(np.array([1, 2, 3]), 'ft', 'in') # → array([12, 24, 36])
9
10 # Complex units
11 convert(50, 'kg/m3', 'lb/ft3') # → 3.121
12
13 # Multi-dimensional compound unit (see Chapter 4 walkthrough)
14 import numpy as np
15 rates = np.array([100, 250, 500, 750, 1000])
16 si_rates = convert(rates, 'stb/day/psi', 'm3/s/kPa')
17 # Traverses volume, time, and pressure subgraphs simultaneously
18 # Result cached after first call; subsequent calls are O(1)
19
20 # Algorithm selection
21 convert(1, 'mile', 'mm', algorithm='DFS')

```

8.1.2 units()

Factory function to instantiate appropriate Unit subclass.

Signature:

```
1 units(value, unit_str, name=None)
```

Parameters:

- **value**: Numeric quantity
- **unit_str**: Unit string (determines subclass)
- **name**: Optional descriptive label

Returns: Instance of **Length**, **Mass**, **Time**, **Energy**, **Force**, or **Temperature** subclass

Examples:

```

1 from unyts import units
2
3 length = units(50, 'cm')
4 print(length) # → 50.0 cm
5 print(type(length).__name__) # → Length
6
7 mass = units(10, 'kg', name='sample_weight')
8 print(mass.name) # → sample_weight
9
10 temp = units(25, 'C')
11 print(type(temp).__name__) # → Temperature

```

8.2 Unit Class

Base class for all unit objects. All subclasses (**Length**, **Mass**, **Time**, **Energy**, **Force**, **Temperature**, **Unitless**) inherit these methods.

8.2.1 Attributes

- `.value`: Numeric quantity (int, float, array)
- `.unit`: Unit string (e.g., 'm', 'kg/s²')
- `.name`: Optional descriptive string (default `None`)

Examples:

```
1 from unyts import units
2
3 distance = units(1500, 'm', name='track_length')
4 print(distance.value) # → 1500
5 print(distance.unit) # → m
6 print(distance.name) # → track_length
```

8.2.2 Arithmetic Operations

Units support standard Python operators. Results adopt units of the first operand.

Addition / Subtraction

```
1 from unyts import units
2
3 a = units(100, 'cm')
4 b = units(1, 'm')
5 result = a + b
6 print(result) # → 200.0 cm (converts m to cm, then adds)
7
8 c = units(5, 'ft')
9 d = units(12, 'in')
10 print(c - d) # → 4.0 ft (converts in to ft, then subtracts)
```

Multiplication / Division

Creates compound units:

```
1 force = units(10, 'kg') * units(9.8, 'm/s2')
2 print(force) # → 98.0 kg·m/s2 (Force)
3
4 velocity = units(100, 'm') / units(10, 's')
5 print(velocity) # → 10.0 m/s
6
7 # Dimensional analysis
8 energy = units(50, 'kg') * units(2, 'm/s')**2
9 print(energy) # → 200.0 kg·m2/s2 (Energy)
```

Exponentiation

```
1 area = units(5, 'm')**2
2 print(area) # → 25.0 m2
3
4 volume = units(3, 'ft')**3
5 print(volume) # → 27.0 ft3
```

8.2.3 Comparison Operators

Units automatically convert to common units before comparing:

```
1 from unyts import units
2
3 a = units(100, 'cm')
4 b = units(1, 'm')
5 print(a == b)      # → True (converts to same units)
6 print(a < b)       # → False
7 print(a <= b)      # → True
8
9 c = units(1, 'mile')
10 d = units(1600, 'm')
11 print(c > d)       # → True (1 mile = 1609.34 m)
```

8.2.4 .to() Method

Convert instance to different units, modifying in place:

```
1 from unyts import units
2
3 distance = units(1000, 'm')
4 distance.to('km')
5 print(distance)    # → 1.0 km
6 print(distance.value) # → 1.0
7 print(distance.unit)  # → km
8
9 # Method chaining
10 temp = units(100, 'C').to('F')
11 print(temp)       # → 212.0 °F
```

8.3 Module-Level Functions

8.3.1 Configuration

```
1 import unyts
2
3 # Algorithm selection
4 unyts.set_algorithm('BFS') # 'BFS', 'lean_BFS', 'DFS', 'hybrid_BFS'
5 current = unyts.get_algorithm()
6
7 # Parallelization
8 unyts.set_parallel('t') # 't' (threading), 'p' (multiprocessing), False, None
9 unyts.set_parallel(False) # Disable
10
11 # Search timeout
12 unyts.set_timeout(10) # seconds, None for unlimited
13
14 # Logging
15 unyts.verbose(True) # Enable verbose output
16 unyts.verbose(False) # Disable
17
18 # Network rebuild
19 unyts.reload() # Force rebuild from database
```

8.3.2 Memory Management

```
1 from unyts.database import save_memory, load_memory, clean_memory
2
3 # Cache operations
4 save_memory()          # Persist to default location
5 save_memory('custom.cache') # Export to file
6 load_memory()          # Reload from default
7 load_memory('custom.cache') # Import from file
8 clean_memory()         # Clear cache from RAM
```

8.3.3 Path Inspection

Visualize conversion paths:

```
1 import unyts
2
3 unyts.print_path(True)  # Enable path printing
4 result = unyts.convert(100, 'cm', 'in')
5 # Output: [Unyts INFO]: converting from 'cm' to 'in':
6 #          cm x0→.3937 in
7
8 unyts.print_path(False) # Disable
```

8.3.4 Custom Units and Conversions

Add New Unit

```
1 from unyts.database import units_network
2
3 # Define unit with conversion to existing unit
4 units_network.set_unit('smoot', 1.7018, 'm')
5 # 1 smoot = 1.7018 meters
6
7 result = unyts.convert(10, 'smoot', 'ft')
8 # Now 'smoot' is available in all conversions
```

Add Direct Conversion

```
1 from unyts.database import units_network
2
3 # Add conversion factor between two existing units
4 units_network.set_conversion('furlong', 'km', 0.201168)
5 # Sets: 1 furlong = 0.201168 km
6
7 # Creates bidirectional edge in graph
8 result = unyts.convert(5, 'km', 'furlong')
```

8.3.5 Formation Volume Factor (FVF)

For petroleum engineering:

```

1 from unyts.database import units_network
2
3 # Set FVF (reservoir vol / surface vol)
4 units_network.set_fvf(1.25) # dimensionless vol/vol
5
6 # Now enables conversions:
7 result = unyts.convert(1000, 'rb', 'sm3') # reservoir barrel → surface m³

```

8.3.6 Constant Density

Bridge volume and mass:

```

1 import unyts
2
3 # Set density for volume ↔ mass conversions
4 unyts.set_density(997) # g/cm³ (water at 25°C)
5
6 # Now enables:
7 mass = unyts.convert(1, 'liter', 'kg') # → 0.997 kg
8 volume = unyts.convert(500, 'g', 'ml') # → ~500.5 ml

```

8.4 Type Checking and Subclasses

```

1 from unyts import units
2 from unyts.units import Length, Mass, Temperature
3
4 distance = units(100, 'm')
5 weight = units(50, 'kg')
6
7 # Type checking
8 isinstance(distance, Length) # → True
9 isinstance(weight, Mass) # → True
10 isinstance(weight, Length) # → False
11
12 # Direct instantiation (alternative to units())
13 temp = Temperature(25, 'C')
14 force = Force(100, 'N')

```

8.5 NumPy and Pandas Integration

Unyts seamlessly handles array-like values:

```

1 import numpy as np
2 import pandas as pd
3 from unyts import convert, units
4
5 # NumPy arrays
6 arr = np.array([1, 2, 3, 4, 5])
7 result = convert(arr, 'km', 'm')
8 print(result) # → [1000. 2000. 3000. 4000. 5000.]
9
10 # Pandas Series

```

```
11 series = pd.Series([10, 20, 30], name='distances')
12 converted = convert(series, 'mile', 'km')
13 print(converted)
14 # 0    16.09
15 # 1    32.19
16 # 2    48.28
17 # Name: distances, dtype: float64
18
19 # Unit objects with arrays
20 temps = units(np.array([0, 25, 100]), 'C')
21 temps.to('F')
22 print(temps.value) # → [ 32.  77. 212.]
```

8.6 Uncertain Units

For measurements with uncertainty:

```
1 from unyts import units
2
3 # Create uncertain unit
4 length = units("100±5", "cm", name="measurement")
5 print(length) # → (100±5) cm
6
7 # Propagates through arithmetic
8 doubled = length * 2
9 print(doubled) # → (200±10) cm
10
11 # Conversion preserves uncertainty
12 in_meters = length.to('m')
13 print(in_meters) # → (1.0±0.05) m
```

Chapter 9

Troubleshooting

9.1 Installation Issues

9.1.1 Module Not Found

Problem: `ModuleNotFoundError: No module named 'unyts'`

Solution:

1. Verify installation: `pip list | grep unyts`
2. Reinstall: `pip install --upgrade --force-reinstall unyts`
3. Check Python environment (virtual env vs system)
4. Ensure pip installs to correct Python interpreter

9.1.2 Dependency Conflicts

Problem: Package version conflicts during installation

Solution:

```
1 # Create clean virtual environment
2 python -m venv unyts_env
3 source unyts_env/bin/activate # Linux/Mac
4 unyts_env\Scripts\activate    # Windows
5
6 pip install unyts
```

9.2 Conversion Errors

9.2.1 NoConversionFoundError

Problem: `unyts.errors.NoConversionFoundError: No conversion found between 'X' and 'Y'`

Causes and Solutions:

1. Incompatible dimensions: Cannot convert length to mass without density

```
1 # Won't work: different dimensions
2 convert(10, 'kg', 'm') # X Mass ≠ Length
3
4 # Solution: set density bridge
5 unyts.set_density(1000) # kg/m³
6 convert(10, 'kg', 'liter') # ✓ Mass → Volume via density
```


2. Typo in unit name:

```
1 convert(5, 'kilogram', 'g') # ✗ Use 'kg' not 'kilogram'
2 convert(5, 'kg', 'g')      # ✓ Correct abbreviation
```

3. Custom unit not defined:

```
1 # Define custom unit first
2 from unyts.database import units_network
3 units_network.set_unit('smoot', 1.7018, 'm')
4 convert(10, 'smoot', 'ft') # ✓ Now works
```

4. Disconnected graph component: Unit exists but has no path to target

```
1 # Add conversion bridge
2 units_network.set_conversion('unit_a', 'unit_b', factor)
```

9.2.2 SearchTimeoutError

Problem: `unyts.errors.SearchTimeoutError`: Search exceeded timeout limit

Solutions:

1. Increase timeout:

```
1 unyts.set_timeout(30) # 30 seconds
2 # or
3 convert(value, from_unit, to_unit, timeout=30)
```

2. Disable timeout:

```
1 unyts.set_timeout(None) # Unlimited time
```

3. Try faster algorithm:

```
1 unyts.set_algorithm('DFS') # Often faster for complex graphs
```

4. Enable caching: Cache results to avoid repeated searches

```
1 unyts.cache(True)
2 convert(value, from_unit, to_unit) # First call: slow search
3 convert(value2, from_unit, to_unit) # Second call: instant (cached)
```

9.2.3 NoFVFEError

Problem: `unyts.errors.NoFVFEError`: FVF must be set for reservoir/surface volume conversions

Solution:

```
1 from unyts.database import units_network
2
3 # Set formation volume factor
4 units_network.set_fvf(1.25) # vol/vol
5
6 # Now works:
7 convert(1000, 'rb', 'sm3') # Reservoir barrel → surface m³
```

9.2.4 WrongFormatError

Problem: `unyts.errors.WrongFormatError: Invalid unit format`

Common causes:

- Spaces in compound units: use `'kg/m3'` not `'kg / m3'`
- Unicode issues: ensure UTF-8 encoding for symbols like `'°C'`
- Unsupported operators: use `/` or `·` for division/multiplication

9.3 GUI Issues

9.3.1 GUI Won't Launch

Problem: `python -m unyts` fails or hangs

Solutions:

1. Check Tkinter installed:

```
1 python -c "import tkinter" # Should not error
```

2. On Linux, install Tkinter:

```
1 # Ubuntu/Debian
2 sudo apt-get install python3-tk
3
4 # Fedora/RHEL
5 sudo dnf install python3-tkinter
```

3. Verify display server (Linux):

```
1 echo $DISPLAY # Should output :0 or similar
```

9.3.2 GUI Crashes on Conversion

Problem: GUI closes or freezes during conversion

Solutions:

- Disable parallel mode: Options → Parallel → off
- Reduce timeout: Options → Search algorithm → Set timeout → 5 seconds
- Clear cache: File → Delete cache files, then restart
- Check console for error messages (run from terminal)

9.4 Performance Issues

9.4.1 Slow First Conversion

Cause: First run builds units network from database definitions (typically 2-5 seconds)

Solutions:

- Enable caching: `unyts.cache(True)` — subsequent runs load from cache instantly
- Pre-build cache: Run `python -m unyts` once to generate cache
- Cache persists across sessions in user folder

9.4.2 Slow Subsequent Conversions

Causes and solutions:

1. Complex path: Many graph hops required

```
1 # Add direct conversion shortcut
2 from unyts.database import units_network
3 units_network.set_conversion('unit_a', 'unit_b', factor)
```

2. Inefficient algorithm:

```
1 unyts.set_algorithm('lean_BFS') # Fastest for most cases
2 # or
3 unyts.set_algorithm('DFS')      # Fast for dense graphs
```

3. Cache disabled:

```
1 unyts.cache(True) # Enable persistent caching
```

9.5 Cache Corruption

Symptoms: Incorrect conversion results, crashes, or “file corrupt” errors

Solution:

```
1 from unyts.database import delete_cache
2 delete_cache() # Remove all .cache files
3
4 # Or manually delete from:
5 # Windows: C:\Users\<user>\AppData\Local\unyts\
6 # Linux/Mac: ~/.unyts/
7
8 unyts.reload() # Force rebuild
```

9.6 Logging and Debugging

9.6.1 Enable Verbose Mode

See detailed search paths and timing:

```
1 unyts.verbose(True)
2 convert(100, 'cm', 'in')
3 # Output:
4 # [Unyts INFO]: converting from 'cm' to 'in': cm ×0.3937 in
5 # [Unyts DEBUG]: Search completed in 0.0012s
```

9.6.2 Adjust Logging Level

```
1 from unyts.helpers.logger import logger
2
3 logger.set_level('DEBUG') # Most verbose
4 logger.set_level('INFO')  # Default
5 logger.set_level('WARNING') # Errors only
```

9.6.3 Inspect Network Structure

```
1 from unyts.database import units_network
2
3 # View all units
4 all_units = units_network.get_all_units()
5 print(f"Total units: {len(all_units)}")
6
7 # Check if unit exists
8 if 'smoot' in all_units:
9     print("Unit 'smoot' is defined")
10
11 # View conversion graph statistics
12 graph = units_network.get_graph()
13 print(f"Nodes: {len(graph.nodes)}, Edges: {len(graph.edges)}")
```

9.7 Getting Help

- GitHub Issues: <https://github.com/ayaranitram/unyts/issues>
- Demo Notebook: https://github.com/ayaranitram/unyts/blob/master/unyts_demo.ipynb
- Documentation: Browse source code docstrings for detailed API reference
- Email: Contact maintainer Martín Carlos Araya at martinaraya@gmail.com

When reporting issues, include:

1. Unyts version: `python -c "import unyts; print(unyts.__version__)"`
2. Python version: `python --version`
3. Operating system
4. Minimal code example reproducing the problem
5. Full error traceback

Appendix A

Comprehensive Unit Reference

This appendix lists all recognized units in Unyts, grouped by physical quantity type and sorted alphabetically within each category.

A.1 Length Units

SI Metric: angstrom, centimeter (cm), decimeter (dm), dekameter (dam), hectometer (hm), kilometer (km), meter (m), micrometer (μm), millimeter (mm), nanometer (nm), picometer (pm)

Imperial: chain (ch), fathom, foot (ft), furlong, inch (in), league, mile (mi), nautical mile, rod, thou, yard (yd)

Specialized: astronomical unit (AU), light-year (ly), parsec, point, pica

A.2 Mass Units

SI Metric: gram (g), kilogram (kg), microgram (μg), milligram (mg), nanogram (ng), tonne, metric ton

Imperial: carat, dram, grain, ounce (oz), penny weight, pound (lb), pound-force (lbf), stone, ton (US), ton (UK)

Atomic: atomic mass unit (u, amu)

A.3 Time Units

SI Metric: attosecond, centisecond, day, decisecond, femtosecond, hour (h), microsecond (μs), millisecond (ms), minute (min), nanosecond (ns), picosecond (ps), second (s)

Calendar: century, decade, lustrum, month (mo), week (w, we), year (y, yr)

A.4 Temperature Units

Absolute: Kelvin (K), Rankine (R)

Relative: Celsius (C, $^{\circ}\text{C}$), Fahrenheit (F, $^{\circ}\text{F}$)

Temperature Gradients: K/m, $^{\circ}\text{C}/\text{m}$, $^{\circ}\text{C}/100\text{m}$, $^{\circ}\text{F}/100\text{ft}$, $^{\circ}\text{F}/\text{ft}$

A.5 Volume Units

SI Metric: cubic centimeter (cm^3 , cc, ml), cubic decimeter (dm^3), cubic meter (m^3), liter (L, l)

Imperial (US): barrel (bbl), bushel, cubic foot (ft³), cubic inch (in³), cubic yard (yd³), cup, fluid ounce (fl oz), gallon (gal), gill, pint (pt), quart (qt), teaspoon (tsp), tablespoon (tbsp)

Imperial (UK): imperial barrel, imperial bushel, imperial cup, imperial fluid ounce, imperial gallon, imperial pint, imperial quart

Oil Industry: stock tank barrel (stb), reservoir barrel (rb)

A.6 Pressure Units

SI Metric: bar, millibar (mbar), pascal (Pa), kilopascal (kPa), megapascal (MPa)

Imperial: atmosphere (atm), psi (pound per square inch), psf (pound per square foot)

Other: mmHg (millimeter of mercury), torr, dyne/cm²

A.7 Pressure Gradient Units

Common petroleum measurements: psi/foot, psi/meter, bar/foot, bar/meter, Pa/meter

A.8 Area Units

SI Metric: square centimeter (cm²), square decimeter (dm²), hectare (ha), square kilometer (km²), square meter (m²), square millimeter (mm²)

Imperial: acre, square chain, square foot (ft²), square inch (in²), square mile (mi²), square rod, square yard (yd²)

A.9 Density Units

SI/Metric: g/cm³, g/ml, kg/m³, kg/L, metric ton/m³

Imperial: lb/ft³, lb/gallon (lb/gal), lb/in³

Oil industry: API degrees (°API)

A.10 Velocity / Speed Units

SI Metric: centimeter per second (cm/s), kilometer per hour (km/h), meter per second (m/s)

Imperial: foot per second (ft/s), mile per hour (mph), knot (nautical mile per hour)

A.11 Rate Units

Volume flow: barrel per day (bbl/day), cubic meter per second (m³/s), cubic meter per hour (m³/h), liter per second (L/s), gallon per minute (gpm)

Mass flow: kilogram per second (kg/s), metric ton per hour (ton/h)

A.12 Force Units

SI Metric: dyne, kilonewton (kN), millinewton (mN), newton (N)

Imperial: kilogram-force (kgf), pound-force (lbf), poundal

A.13 Energy / Work / Heat Units

SI Metric: calorie (cal), foot-pound, joule (J), kilocalorie (kcal), kilowatt-hour (kWh), megajoule (MJ), millijoule (mJ), watt-hour (Wh)

Imperial: British thermal unit (BTU)

Energy equivalent: barrel of oil equivalent (boe), ton of TNT

A.14 Power Units

SI Metric: kilowatt (kW), megawatt (MW), milliwatt (mW), watt (W)

Imperial: horsepower (hp), foot-pound per second (ft-lbf/s)

A.15 Viscosity Units

Dynamic: centipoise (cP), poise, pascal-second (Pa·s)

Kinematic: centistoke (cSt), stoke (St), square meter per second (m²/s)

A.16 Permeability Units

Darcy system: darcy, millidarcy (mD), microdarcy (μD)

SI system: square meter, square micrometer

A.17 Data Units

Bytes: byte (B), kilobyte (KB), megabyte (MB), gigabyte (GB), terabyte (TB), petabyte (PB)

Bits: bit (b), kilobit (kb), megabit (Mb), gigabit (Gb)

A.18 Dimensionless Quantities

Ratios and factors: percent (%), parts per million (ppm), parts per billion (ppb)

Petroleum-specific: API degree (°API, dimensionless measure of crude oil gravity)

A.19 Notes on Unit Selection

A.19.1 SI vs. Imperial Choice

- SI (metric) units are recommended for scientific applications and are the international standard
- Imperial units are common in aerospace, petroleum, and US-based engineering
- Unyts supports both seamlessly—use whichever is appropriate for your application
- When mixing systems, conversions route through defined bridges (e.g., meter ↔ yard for length)

A.19.2 Petroleum Abbreviations

Special note to petroleum engineers:

- bbl: Stock tank barrel (one standard definition)
- rb: Reservoir barrels (requires FVF setting for conversion)
- sm3: Surface cubic meters (stock tank equivalent)
- rm3: Reservoir cubic meters
- Ksm3: Thousand surface cubic meters
- BBL/day: Barrels per day (rate unit)
- mD: Millidarcy (permeability, crucial for flow calculations)

A.19.3 Temperature Notes

Temperature conversions are more complex than simple unit conversions:

- Absolute scales (Kelvin, Rankine): Can be converted directly
- Relative scales (Celsius, Fahrenheit): Have offsets; conversion is not just a factor
- Temperature differences/gradients: Always use Kelvin or Rankine intervals; °C/K differences are equivalent

Example:

```

1 # Correct: Temperature value (includes offset)
2 unyts.convert(0, 'C', 'F') # → 32 °F (freezing point)
3
4 # Incorrect: Treating Celsius difference as temperature
5 # A 100°C temperature range = 180°F range (different!)
6 # For differences, use Kelvin intervals
7
8 # Correct: Temperature difference
9 # A 100 K difference = 180 R difference
10 unyts.convert(100, 'K/s', 'R/s') # Rate of temperature change

```

A.20 Extending the Unit Database

To add custom units or new unit definitions to your Unyts installation:

```

1 from unyts.database import units_network
2
3 # Define a new unit relative to an existing one
4 units_network.set_unit('smoots', 5.1829, 'foot')
5 # Now 'smoot' is recognized in all conversions
6
7 # Add direct conversion between units
8 units_network.set_conversion('furlong', 'hectometer', 0.20117)
9
10 # Persist for next session
11 units_network.set_fvf(1.25) # Petroleum FVF
12 unyts.set_density(850) # Custom density in kg/m³

```

See the Python API Reference chapter for detailed API documentation.

Appendix B

Foundational Conversion Definitions

The table below lists every foundational (single-hop) conversion edge that is hard-coded in the directed graph built by `database.py`. Every other conversion that Unyts can perform is derived by composing a chain of these edges via the search algorithms described in the Search Algorithms chapter.

Conversions marked $y = x$ are dimensional aliases—the two unit names share the same scale and offset (e.g. `bara` $\equiv$ `bar`). Prefix-based conversions (`kilo-`, `milli-`, etc.) and automatically generated inverse edges are not listed here; they are created programmatically at import time.

Constants used in formulas:

- $g = 9.80665 \text{ m/s}^2$ (standard Earth gravity)
- $c = 299\,792\,458 \text{ m/s}$ (speed of light)
- $\rho_{\text{air}} = 1.225 \text{ kg/m}^3$ (standard air density)
- FVF: formation volume factor, user-configurable via `set_fvf()`

Table B.1: Foundational conversion edges in the Unyts directed graph.

From	To	Formula
Percentage & Fraction		
fraction	percentage	$y = x \times 100$
percentage	fraction	$y = x \div 100$
Time		
second	millisecond	$y = x \times 1000$
minute	second	$y = x \times 60$
hour	minute	$y = x \times 60$
day	hour	$y = x \times 24$
day	month	$y = x \div 365.25 \times 12$
week	day	$y = x \times 7$
year	month	$y = x \times 12$
year	day	$y = x \times 365.25$
lustrum	year	$y = x \times 5$
decade	year	$y = x \times 10$
century	year	$y = x \times 100$
Temperature		
continued on next page...		

From	To	Formula
Celsius	Kelvin	$y = x + 273.15$
Kelvin	Celsius	$y = x - 273.15$
Celsius	Fahrenheit	$y = x \times 9/5 + 32$
Fahrenheit	Celsius	$y = (x - 32) \times 5/9$
Fahrenheit	Rankine	$y = x + 459.67$
Rankine	Fahrenheit	$y = x - 459.67$
Rankine	Kelvin	$y = x \times 5/9$
Kelvin	Rankine	$y = x \times 9/5$
Length		
yard	meter	$y = x \times 0.9144$
inch	thou	$y = x \times 1000$
inch	tenth	$y = x \times 10$
foot	inch	$y = x \times 12$
yard	foot	$y = x \times 3$
chain	yard	$y = x \times 22$
furlong	chain	$y = x \times 10$
mile	furlong	$y = x \times 8$
league	mile	$y = x \times 3$
nautical league	nautical mile	$y = x \times 3$
nautical mile	meter	$y = x \times 1852$
rod	yard	$y = x \times 5.5$
astronomical unit	meter	$y = x \times 149\,597\,870\,700$
parsec	astronomical unit	$y = x \times 206\,265$
light year	meter	$y = x \times c \times 365.25 \times 24 \times 60$
scandinavian mile	kilometer	$y = x \times 10$
Velocity		
mile per hour	kilometer per hour	$y = x \times 1.609344$
Area		
square kilometer	square meter	$y = x \times 10^6$
square mile	acre	$y = x \times 640$
acre	square yard	$y = x \times 4840$
square rod	square yard	$y = x \times 30.25$
square yard	square foot	$y = x \times 9$
square foot	square inch	$y = x \times 144$
square foot	square meter	$y = x \times 0.3048^2$
square inch	square thou	$y = x \times 10^6$
square inch	square tenth	$y = x \times 100$
square chain	square yard	$y = x \times 22^2$
square furlong	square chain	$y = x \times 10^2$
square mile	square furlong	$y = x \times 8^2$
square league	square mile	$y = x \times 3^2$
Permeability		
Darcy	μm^2	$y = x \times 0.9869233$

continued on next page...

From	To	Formula
Volume		
litre	cubic centimeter	$y = x \times 1000$
gill	fluid ounce	$y = x \times 4$
pint	gill	$y = x \times 4$
quart	pint	$y = x \times 2$
gallonUS	fluid ounce	$y = x \times 128$
gallonUS	quart	$y = x \times 4$
gallonUS	cubic inch	$y = x \times 231$
gallonUK	quartUK	$y = x \times 4$
gallonUK	fluid ounce UK	$y = x \times 160$
gallonUK	litre	$y = x \times 4.54609$
gillUK	fluid ounce UK	$y = x \times 4$
pintUK	gillUK	$y = x \times 4$
quartUK	pintUK	$y = x \times 2$
gallonUK	liter	$y = x \times 4.54609$
cubic foot	cubic meter	$y = x \times 0.3048^3$
std cubic foot	std cubic meter	$y = x \times 0.3048^3$
std barrel	USgal	$y = x \times 42$
std cubic meter	std barrel	$y = x \times 6.289814$
std barrel	std cubic foot	$y = x \times 5.614584$
res cubic meter	res barrel	$y = x \times 6.289814$
res cubic meter	std cubic meter	$y = x \div \text{FVF}^\dagger$
cubic inch	cubic thou	$y = x \times 10^9$
cubic inch	cubic tenth	$y = x \times 10^3$
cubic foot	cubic inch	$y = x \times 12^3$
cubic yard	cubic foot	$y = x \times 3^3$
cubic chain	cubic yard	$y = x \times 22^3$
cubic furlong	cubic chain	$y = x \times 10^3$
cubic mile	cubic furlong	$y = x \times 8^3$
cubic league	cubic mile	$y = x \times 3^3$
Pressure		
psi gauge	absolute psi	$y = x + 14.6959$
absolute psi	psi gauge	$y = x - 14.6959$
bar gauge	absolute bar	$y = x + 1.01325$
absolute bar	bar gauge	$y = x - 1.01325$
absolute bar	absolute psi	$y = x \times 14.504$
bar gauge	psi gauge	$y = x \times 14.504$
bar	psi	$y = x \times 14.504$
psi	bar	$y = x \div 14.504$
absolute bar	Pascal	$y = x \times 100\,000$
atmosphere	Pascal	$y = x \times 101\,325$
atmosphere	Torr	$y = x \times 760$
absolute bar	bar	$y = x$
absolute psi	psi	$y = x$
bar gauge	bar	$y = x$
psi gauge	psi	$y = x$

continued on next page...

From	To	Formula
absolute bar	kg/cm ²	$y = x \times 10 \div g$
Mass		
grain	milligrams	$y = x \times 64.7989$
pennyweight	grain	$y = x \times 24$
dram	pound	$y = x \div 256$
stone	pound	$y = x \times 14$
quarter	stone	$y = x \times 2$
weight ounce	dram	$y = x \times 16$
pound	weight ounce	$y = x \times 16$
long hundredweight	quarter	$y = x \times 4$
short hundredweight	pound	$y = x \times 100$
short ton	short hundredweight	$y = x \times 20$
long ton	long hundredweight	$y = x \times 20$
metric ton	kilogram	$y = x \times 1000$
kilogram	gram	$y = x \times 1000$
pound	kilogram	$y = x \times 0.45359237$
Force		
kilogram mass	kilogram force	$y = x \times g$
kilogram force	kilogram mass	$y = x \div g$
Dyne	Newton	$y = x \times 10^{-5}$
Newton	Dyne	$y = x \times 10^5$
pound force	kilogram force	$y = x \times 0.45359237$
kilogram force	Newton	$y = x \times g$
Energy		
Joule	gram calorie	$y = x \div 4.184$
Kilojoule	Joule	$y = x \times 1000$
Kilojoule	kilowatt hour	$y = x \div 3600$
Kilojoule	British thermal unit	$y = x \div 1.055$
Power		
Horsepower	Watt	$y = x \times 745.699872$
Density		
API	SgO	$y = 141.5 \div (131.5 + x)$
SgO	API	$y = 141.5 \div x - 131.5$
API	g/cc	$y = 141.5 \div (131.5 + x)$
g/cc	API	$y = 141.5 \div x - 131.5$
SgO	g/cc	$y = x$
SgW	g/cc	$y = x$
SgG	kg/m ³	$y = x \times \rho_{\text{air}}$
psia/ft	lb/ft ³	$y = x \times 144$
psi/ft	lb/ft ³	$y = x \times 144$
psig/ft	lb/ft ³	$y = x \times 144$
bara/m	kg/m ³	$y = x \times 100\,000 \div g$
bar/m	kg/m ³	$y = x \times 100\,000 \div g$

continued on next page...

From	To	Formula
barg/m	kg/m ³	$y = x \times 100\,000 \div g$
g/cm ³	lb/ft ³	$y = x \times 62.428$
lb/ft ³	lb/stb	$y = x \times 5.614584$
Viscosity		
Pascal·second	Poise	$y = x \times 10$
Pascal·second	Reyn	$y = x \times 1.4504 \times 10^{-4}$
Pascal·second	Poiseuille	$y = x$
Data		
byte	bit	$y = x \times 8$

† The FVF (formation volume factor) is a user-configurable parameter set via `set_fvf()`. It defaults to **1.0** (identity). See the Python API Reference chapter for details.